GPU Micromagnetic Simulation with Irregular Geometry

Qianru Wu
Carnegie Mellon University

March 2026

We use multiple problem sizes in different sections to highlight different performance regimes, including overhead-dominated and bandwidth-dominated behavior.

Project URL

Project page link: https://angelawoo572.github.io/gpu-micromagnetics-irregular/

Summary

We study the end-to-end performance of a GPU-accelerated micromagnetic simulation system for the Landau–Lifshitz–Gilbert (LLG) equation using CUDA and SUNDIALS/CVODE. While each RHS evaluation is spatially data-parallel, the full simulation is controlled by an adaptive CVODE-driven time-integration pipeline, making it unclear whether GPU kernel tuning or solver-side overhead is the real performance bottleneck.

We first build a fully working 2D local-interaction solver and analyze regular-grid performance through layout, block-shape, tolerance, scaling, and Nsight profiling experiments. These experiments show that kernel-level optimizations provide only modest gains, while solver orchestration, synchronization, and repeated SUNDIALS vector operations dominate end-to-end runtime.

We then extend the solver to irregular masked geometries and implement both dense masked and compact active-cell execution. Finally, motivated by the profiling results, we explore solver-level optimizations including a fused NVector layer, a block-diagonal preconditioner, and an analytic Jacobian–vector product. Together, these results show that optimizing a CVODE-driven GPU simulation requires end-to-end pipeline reasoning, not only faster physics kernels.

Introduction

Micromagnetic simulation for the Landau–Lifshitz–Gilbert (LLG) equation leads to a large coupled system of nonlinear ordinary differential equations, with three unknowns per spatial cell. A GPU is a natural platform for this workload because each right-hand-side (RHS) evaluation exposes substantial spatial parallelism across cells.

However, the full simulation is not embarrassingly parallel. In our system, time integration is handled by SUNDIALS/CVODE, so end-to-end runtime depends not only on the cost of the user-defined GPU RHS kernel, but also on adaptive step control, nonlinear solves, repeated vector operations, and synchronization across the solver pipeline. This raises the main systems question

of the project: *what actually dominates performance in a CVODE-driven GPU micromagnetic simulation?*

To answer this question, we first study a regular-grid 2D local-interaction solver and evaluate the effects of data layout, block shape, tolerance, problem size, and execution breakdown. We then extend the solver to irregular masked geometries and compare dense masked versus compact active-cell execution. Finally, motivated by profiling evidence that solver-side overhead dominates runtime, we implement solver-level optimizations targeting synchronization and linear-algebra costs directly. The overall result is a clearer end-to-end picture of how GPU parallelism, irregular geometry, and adaptive time integration interact in this class of simulations.

System Overview

We discretize the domain on a 2D grid, with one 3-component magnetization vector (m_x, m_y, m_z) per cell. This leads to a system with three unknowns per grid cell.

Each RHS evaluation computes local nearest-neighbor interactions, which can be evaluated independently across cells and is therefore well suited for GPU parallelization.

The system uses CUDA kernels for RHS evaluation and SUNDIALS CVODE (BDF) for time integration, with GPU-resident vectors (N_Vector CUDA).

Unless otherwise noted, we focus on a local-interaction-only 2D solver (no FFT) as a baseline for performance analysis.

Baseline End-to-End Analysis on the Regular Grid

All experiments in this section use a baseline grid size of $n_x = 900$, $n_y = 128$ (unless otherwise specified). We perform three main experiments:

1. Data layout comparison (AoS vs. SoA)
2. Block configuration sweep
3. Solver tolerance sweep

All results report total **GPU time-to-solution**, since CVODE introduces sequential dependencies across timesteps.

Kernel-Level Tuning: Layout and Block Shape

We first evaluate two standard GPU kernel-level tuning choices: data layout and thread-block configuration.

Layout: AoS vs. SoA. We compare two layouts:

- **AoS (Array of Structures):** (m_x, m_y, m_z) stored per cell
- **SoA (Structure of Arrays):** separate arrays for m_x, m_y, m_z

Layout	Time (ms)	Speedup
AoS	~ 14796	1.00x
SoA	~ 14733	~ 1.004x

SoA provides a small but consistent improvement. This is expected because SoA improves memory coalescing when adjacent threads access the same component. However, since each thread still needs all three components, the overall benefit is limited.

Block shape sweep. We next evaluate different thread-block shapes and find that the best configuration is **BLOCK_X = 16, BLOCK_Y = 8**. This choice provides the best balance among occupancy, memory coalescing, and warp utilization.

Different block shapes produce measurable differences, but only within a limited range of a few percent.

Takeaway. These results show that conventional GPU kernel-level tuning is useful, but only provides modest end-to-end gains in this CVODE-driven setting.

Solver-Level Sensitivity: Tolerance

We next study solver tolerance, which changes CVODE's adaptive behavior and therefore can alter the total amount of solver work.

We test:

- 10^{-4}
- 10^{-5}
- 10^{-6}

Tolerance	Time (ms)	ncfn	netf
10^{-4}	**13415**	553	1
10^{-5}	14560	0	1
10^{-6}	13934	558	6

Observation.

- 10^{-4} gives the fastest runtime
- 10^{-5} is the most stable (no convergence failures)
- 10^{-6} increases solver work without benefit

Interpretation. Relaxing tolerance reduces the amount of effective solver work, but it can also introduce nonlinear convergence failures.

Takeaway. Compared with layout and block-shape tuning, tolerance has a much larger effect on total runtime. This is the first strong indication that solver-side behavior is a more important performance lever than kernel-level tuning alone.

Problem-Size Scaling and Nsight Analysis

To understand when the regular 2D local solver transitions from overhead-dominated execution to more substantial GPU work, we fix the best-performing regular-grid configuration (SoA layout, 16×8 blocks, tolerance 10^{-4}) and vary problem size by increasing n_x while holding $n_y = 128$ fixed.

n_x	Total cells	Time (ms)
96	4096	6823.372
192	8192	7032.230
384	16384	7100.213
768	32768	7795.561
1536	65536	8990.308
3072	131072	9941.907
6144	262144	11180.528
12288	524288	16812.521

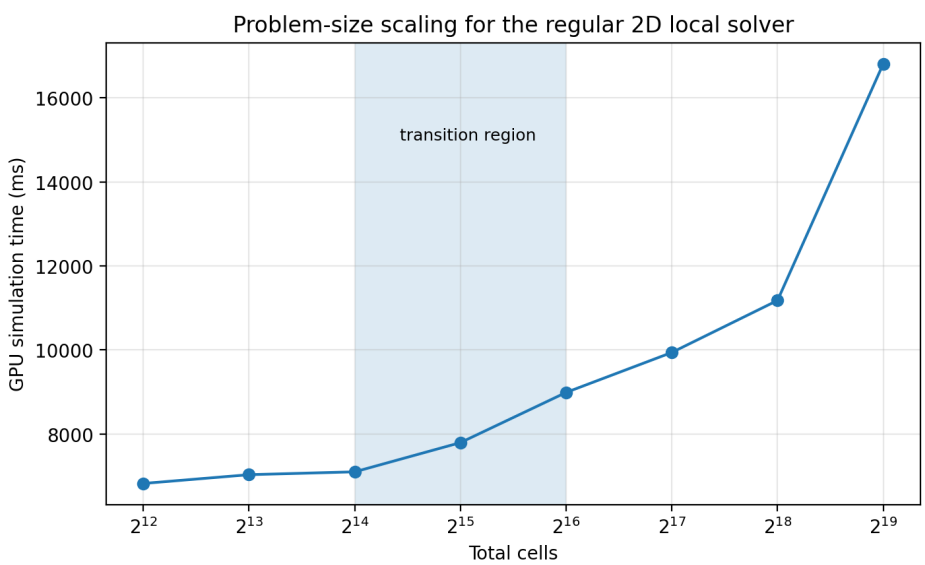

Figure 1: Problem-size scaling of the regular 2D local solver. The smallest cases are nearly flat, indicating fixed overhead dominance. Runtime increases more clearly beyond the small-size regime, with a transition around 1.6×10^4 to 6.6×10^4 cells.

Observation. The smallest cases are nearly flat: increasing the problem from 4096 to 16384 cells changes runtime only from about 6.82 s to 7.10 s. Beyond this range, runtime begins to increase more clearly, reaching about 16.8 s at 524288 cells.

Interpretation. This behavior reveals three execution regimes:

- **Small-size regime:** runtime is dominated by fixed overhead such as solver orchestration, synchronization, and repeated vector kernels.
- **Transition regime:** both overhead and GPU computation contribute significantly to runtime.

- **Large-size regime:** runtime increases with problem size as the GPU workload becomes more substantial.

To better understand these regimes, we profile three representative problem sizes using Nsight Systems:

- **Small:** 4096 cells
- **Mid:** 65536 cells
- **Large:** 524288 cells

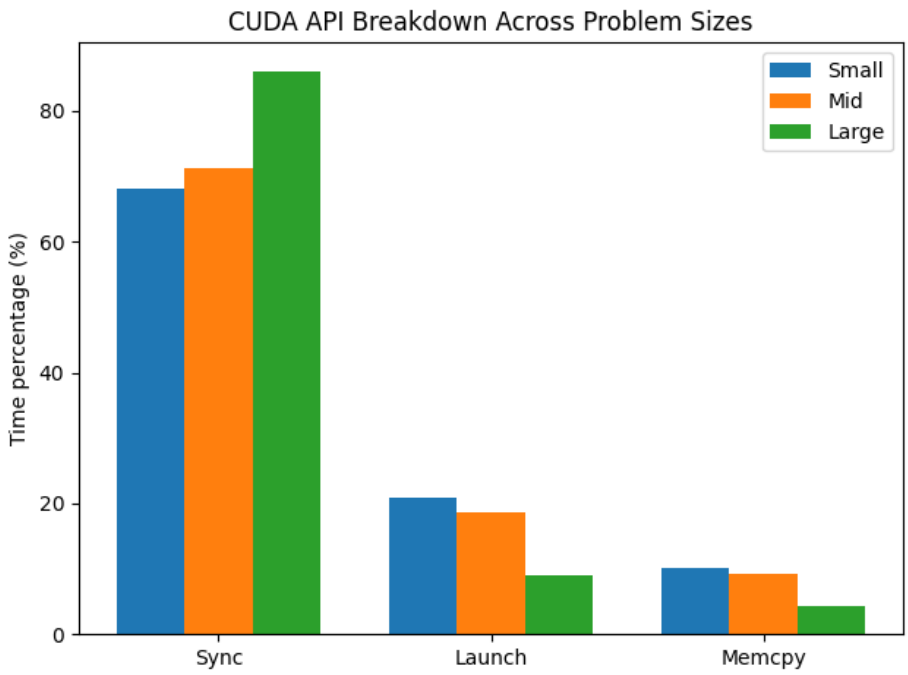

Figure 2: CUDA API time breakdown for small, mid, and large problem sizes. Synchronization dominates across all sizes, while kernel launch and memory copy become relatively less important at larger scales.

Metric	Small	Mid	Large
cudaStreamSynchronize	68.1%	71.2%	86.1%
cudaLaunchKernel	20.9%	18.7%	9.0%
cudaMemcpyAsync	10.1%	9.3%	4.4%
RHS kernel share	7.2%	10.4%	12.1%

Nsight-based explanation. Across all problem sizes, `cudaStreamSynchronize` dominates CUDA API time, indicating that synchronization and solver orchestration are major contributors to total runtime. At small sizes, the RHS kernel accounts for only a small fraction of GPU kernel time, which explains the nearly flat scaling. As the problem size increases, the RHS kernel and SUNDIALS vector kernels become heavier, but synchronization remains a dominant cost.

Takeaway. The main bottleneck is not a single CUDA kernel, but the interaction between CVODE-driven orchestration, repeated SUNDIALS vector operations, and the user-defined RHS kernel. The transition we observe is therefore not a fixed hardware threshold, but the point where larger workloads begin to better amortize solver-side overhead.

Conclusion. Based on these regular-grid experiments, we use SoA layout, 16×8 thread blocks, and tolerance 10^{-4} as the baseline high-performance configuration for subsequent irregular-geometry and solver-level studies. This choice prioritizes speed over robustness, as the fastest setting still exhibits nonzero convergence failures.

More importantly, these results highlight that overall performance is primarily governed by solver-level effects rather than kernel-level tuning. In particular, the observed scaling behavior reflects the extent to which CVODE-driven overhead—including synchronization and repeated SUNDIALS vector operations—is amortized by increasing workload size.

Irregular Geometry as a Systems Stress Test

In this section, we extend the regular 2D local solver to an irregular masked geometry embedded in the same structured grid. Unless otherwise specified, the irregular experiments use the periodic 2D code path with a circular interior hole, and all irregular-geometry experiments use a baseline problem size of $n_x = 1536$, $n_y = 128$.

Dense Masked Baseline

Our irregular domain is built on the same regular 2D grid with periodic boundary conditions in both x and y, so the outer domain is topologically equivalent to a torus. Irregularity is introduced by masking out a circular hole in the interior, so only the remaining active cells evolve.

Our first irregular implementation is a **correctness-first dense masked baseline**. The state remains in SoA layout,

$$[m_x \text{ for all cells}][m_y \text{ for all cells}][m_z \text{ for all cells}],$$

and the full periodic grid is still launched in the RHS kernel.

The execution policy is:

- one CUDA thread per physical cell in the full periodic grid,
- if the current cell is inactive (inside the hole), its derivative is set to zero,
- if a neighbor is inactive, that neighbor contribution is skipped,
- neighbor indexing uses periodic wrap-around in both dimensions.

This design preserves the original CVODE + CUDA structure and isolates the effect of irregular geometry before introducing more aggressive execution strategies.

We verified the irregular implementation through several sanity checks. Setting the hole radius to zero reduces the system to the periodic regular-grid case, providing a direct consistency test.

To further confirm logical consistency, we ran the same irregular code path with `HOLE_RADIUS_FRAC_Y = 0.0`, so that the entire domain remains active.

Case	Active Cells	Time (ms)	nfe	ncfn
Periodic regular ($R = 0$)	65536 / 65536	4425.310	4021	207

This provides a consistent periodic baseline from the irregular code path for later comparison with masked geometries.

We next evaluated the dense masked irregular implementation over several hole sizes. In all these runs, the execution strategy remains a dense full-grid launch, while the circular interior region is masked out.

Hole Radius Fraction	Active %	Time (ms)	nst	nfe	ncfn
0.10	99.20%	8234.460	6816	8015	580
0.16	98.01%	8594.956	6819	8028	584
0.22	96.19%	8789.738	6920	8344	692
0.28	93.83%	8855.327	6551	7760	2
0.34	90.94%	9312.673	6556	7784	2

Observation. Introducing even a modest hole increases end-to-end runtime substantially relative to the $R = 0$ periodic regular case. However, runtime does not scale simply with the number of active cells: larger holes do not always produce proportionally less work or less runtime.

Interpretation. Irregular geometry changes not only GPU execution efficiency, but also the **numerical behavior seen by CVODE**. In particular, the hole modifies local coupling near the irregular boundary, which changes step counts, RHS evaluation counts, and nonlinear solver behavior.

Takeaway. The dense masked irregular case cannot be interpreted purely as "fewer active cells should run faster." In this CVODE-driven workflow, irregular geometry changes both the GPU-side execution pattern and the adaptive solver's amount of work.

Compacted Active-Cell Execution

To reduce wasted work from launching threads over inactive cells, we implemented a second irregular execution strategy: **compacted active-cell execution**.

In this version:

- the global state vector remains in the same dense SoA layout expected by CVODE,
- we build an explicit list of active cell indices,
- the main irregular RHS kernel launches threads only for active cells,
- inactive cells remain in the full state vector but do not evolve,
- a separate kernel sets the inactive entries of $\dot{y}$ to zero.

This allows a direct comparison between two execution strategies on the same irregular geometry:

- **Dense masked execution:** full-grid launch with inactive-cell checks

- **Compact active-cell execution:** launch only over active cells

We compare the two strategies for the same circular hole with radius fraction 0.22.

Execution Strategy	Active %	Time (ms)	nst	nfe	ncfn
Dense masked	96.19%	8789.738	6920	8344	692
Compact active-cell	96.19%	**8464.103**	6921	8347	693

This corresponds to an end-to-end improvement of approximately

$$\frac{8789.738 - 8464.103}{8789.738} \approx 3.7\%.$$

Observation. The CVODE statistics are almost unchanged between the two versions (nst, nfe, and $ncfn$ are essentially identical), which means the numerical problem being solved is the same to first order. The runtime difference is therefore primarily due to **execution organization**, not solver-path changes.

Interpretation. Compaction reduces wasted GPU work by avoiding inactive threads. However, the gain is modest because only 3.81% of the cells are inactive in this geometry, compact execution introduces index indirection, and inactive entries of $\dot{y}$ must still be zeroed explicitly.

Takeaway. Compact active-cell execution is beneficial, but its end-to-end impact is limited for geometries with only a small inactive fraction.

What Irregular Geometry Reveals

Nsight Systems profiling helps explain why compact execution improves runtime only modestly. For the dense masked irregular case ($R = 0.22$), the irregular RHS kernel had an average execution time of approximately $6.92\,\mu s$ per launch. For the compact active-cell version, the compact RHS kernel had an average execution time of approximately $7.19\,\mu s$, and the separate inactive-zeroing kernel added another visible cost.

Thus, the compact RHS kernel is actually slightly slower per launch than the dense masked version. This is expected because compact execution adds active-index indirection, extra integer index reconstruction, and a separate inactive-zeroing kernel. Compact execution still reduces total runtime because it avoids launching the main physics kernel over inactive cells.

At the same time, profiling shows that the custom physics RHS kernel is not the only—or even dominant—source of runtime. For the compact active-cell run, `cudaStreamSynchronize` accounts for roughly 71.6% of CUDA API time, while the custom compact RHS kernel accounts for only about 12.7% of total GPU kernel time. SUNDIALS vector kernels such as `linearSumKernel`, `dotProdKernel`, `scaleKernel`, and `wL2NormSquareKernel` together account for a much larger share.

Interpretation. These results reinforce the main conclusion already suggested by the regular-grid study: end-to-end runtime is dominated by solver-driven orchestration overhead and repeated vector operations, not just by the user RHS kernel alone.

Takeaway. The irregular experiments sharpen the overall systems picture. Irregular geometry changes both GPU execution and solver behavior, dense masked execution provides a simple but wasteful baseline, compact execution gives a real but modest improvement, and the dominant bottleneck remains the larger CVODE-controlled simulation pipeline.

Solver-Level Optimizations

The regular-grid and irregular-geometry studies both show that the dominant bottleneck is not the user RHS kernel alone, but the larger CVODE-driven orchestration loop: repeated vector operations, synchronization, and linear-algebra work inside the Newton–Krylov solver. Optimizing the physics kernel alone therefore cannot remove the main source of end-to-end cost.

Motivated by this result, we next explore a series of solver-level optimizations that target synchronization overhead and repeated linear algebra directly.

Unless otherwise specified, all experiments in this section use a large regular grid with $n_x = 3000$, $n_y = 1280$, $neq = 3{,}840{,}000$, and tolerance 10^{-4}. All timing figures are taken from Nsight Systems profiling runs, which stabilize GPU boost behavior and avoid the large run-to-run variability seen in bare `make run` measurements.

Large-Grid Baseline

We first establish a large-grid baseline in the regime targeted by the solver-level study.

Configuration	Time (ms)	`cudaStreamSync` share
Baseline (SoA, 16×8, tol 10^{-4})	46,249	∼44.1%

Nsight profiling shows that `f_kernel_group_soa_periodic` accounts for only 8.9% of GPU kernel time; most GPU execution time instead lies in SUNDIALS internal vector kernels such as `linearSumKernel`, `dotProdKernel`, and `wL2NormSquareKernel`.

In the full execution trace, `cudaStreamSynchronize` accounts for approximately 44% of CUDA API time, with additional overhead from `cudaLaunchKernel`, `cudaMemcpyAsync`, and initialization costs such as `cudaMalloc`. This confirms that further optimizing the RHS kernel alone would have limited impact. The dominant cost lies in the broader CVODE/SPGMR orchestration layer, including repeated vector operations, synchronization, launch overhead, and associated memory traffic.

Optimization 1: Lazy/Deferred Fused NVector

Motivation. Under CVODE's default *immediate execution* model, every vector operation launches a separate CUDA kernel and blocks the CPU on `cudaStreamSynchronize` to return a scalar. In the small-grid baseline, 291,455 synchronizations occur over the full simulation.

Design. We implement a two-tier fused NVector layer (`deferred_nvector.cu`):

- **Tier 1** calls `N_VEnableFusedOps_Cuda(v, 1)`, activating SUNDIALS' built-in batched vector operations.

- **Tier 3** overrides `nvdotprodmulti` with a custom `multi_dot_kernel`. In SPGMR Classical Gram-Schmidt (CGS) orthogonalization, all K dot products against the current basis vector are independent. Our kernel reads the reference vector x once and accumulates against all K basis vectors in shared memory, producing K scalars with one kernel launch and one `cudaStreamSynchronize` instead of K.

Pointer and coefficient arrays are passed as `struct`-typed kernel arguments (128 bytes in the 4 KB per-launch constant buffer), eliminating H2D memcpy overhead entirely. An earlier implementation used `cudaMemcpy` for these small arrays; it introduced ≈1.6 s of extra synchronous transfer time and was discarded. A **propagating clone** mechanism overrides `nvclone` so that all vectors CVODE allocates internally (Krylov basis, error/weight vectors) inherit the same optimized operations automatically.

Failure mode on the large grid: cache thrashing with CGS. Classical GS triggers `N_VDotProdMulti` and benefits from `multi_dot_kernel`, but its correction step calls `N_VLinearCombination`, which reads all K Krylov vectors simultaneously. For $neq = 3.84 \times 10^6$, each Krylov vector is 30 MB; loading five simultaneously exceeds the L2 cache and reduces effective bandwidth from 88% to 17%. Applying CGS with the `linear_comb_kernel` override on the large grid produced a severe regression:

Configuration	Time (ms)	vs. baseline
Baseline (large grid)	46,249	1.00×
Fused + CGS + `linear_comb_kernel` (large grid)	103,321	0.45×

The runtime more than doubled because the `linear_comb_kernel` alone consumed 54.8 s (>100% of the baseline), far exceeding the sync savings from CGS.

Adaptive GS selection. We implement an automatic regime selection based on *neq*:

- $neq < 500{,}000$ (*overhead-limited*): use Classical GS; install `linear_comb_kernel` override. The problem is too small to saturate GPU bandwidth, so reducing solver-side overhead yields a large end-to-end benefit.
- $neq \geq 500{,}000$ (*bandwidth-limited*): use Modified GS (default); skip the linear-combination override. Each vector operation already runs near bandwidth saturation; cache thrashing from simultaneous multi-vector reads hurts more than saved syncs help.

Results on the small grid (600×128, neq = 76,800).

Configuration	Time (ms)	Speedup
Baseline (small grid)	5,348	1.00×
Fused NVector + CGS	847	**6.3**×

The synchronization count falls by 43% and end-to-end time improves by 6.3× in the overhead-limited regime. These results show that fused execution can be highly effective when solver-side overhead is a large fraction of runtime.

For the large-grid regime, we do not report a fused-only number in isolation, since the fused NVector layer was evaluated together with the later solver-level optimizations.

Optimization 2: Analytic 3×3 Block-Diagonal Jacobi Preconditioner

Motivation. Without a preconditioner, SPGMR requires ≈5.5 GMRES iterations per Newton step. Each iteration involves a full round of vector operations (`linearSumKernel`, `dotProdKernel`, `wL2NormSquareKernel`). Reducing the iteration count would directly reduce the volume of all GMRES-related work, which accounts for ≈60% of GPU kernel time.

Design. For each cell i we derive an analytic 3×3 approximation to the diagonal block $J_{ii} = \partial f_i / \partial m_i$, holding all neighbor magnetizations frozen. The only self-dependent term in the effective field is the easy-axis anisotropy: $\partial h_3 / \partial m_3 = c_{\text{chk}} = 1$. The full 3×3 block is:

$$
\begin{aligned}
&J_{00} = c_\alpha(-h_1 m_1 - \mu), \quad J_{01} = -c_{\text{chg}} h_3 - c_\alpha h_2 m_1, \quad J_{02} = c_{\text{chg}}(h_2 - m_2) - c_\alpha(h_3 + m_3) m_1 \\
&J_{10} = c_{\text{chg}} h_3 - c_\alpha h_1 m_2, \quad J_{11} = c_\alpha(-h_2 m_2 - \mu), \quad J_{12} = c_{\text{chg}}(m_1 - h_1) - c_\alpha(h_3 + m_3) m_2 \\
&J_{20} = -c_{\text{chg}} h_2 - c_\alpha h_1 m_3, \quad J_{21} = c_{\text{chg}} h_1 - c_\alpha h_2 m_3, \quad J_{22} = c_\alpha(1 - (h_3 + m_3) m_3 - \mu)
\end{aligned}
$$

where $\mu = m \cdot H$. This block captures the precession cross-terms $J_{01} = -c_{\text{chg}} h_3$ and $J_{10} = +c_{\text{chg}} h_3$, which a scalar Jacobi preconditioner ignores entirely. We form $P_i = I - \gamma J_i$, invert it analytically via Cramer's rule, and store P_i^{-1} as 9 doubles per cell (≈87 MB total). The `psolve_kernel` applies $z = P^{-1} r$ as a batched 3×3 matrix–vector multiply, one thread per cell.

Bug discovery: incorrect `jok` handling in v1. An initial implementation returned immediately when CVODE passed `jok=SUNTRUE`, reusing the stored P^{-1} without update. Profiling revealed:

Kernel	Calls	Expected
`psetup_kernel` (v1)	266	≈3,753
`psolve_kernel`	20,207	–

CVODE's `jok` protocol guarantees only that J's *structure* is unchanged, not that γ is unchanged. Since $P = I - \gamma J$, every Newton step changes γ and therefore changes P^{-1}. Using a stale P^{-1} means the preconditioner is inconsistent with the current linear system, which explains why GMRES continued to require 5.5 iterations per step.

The corrected implementation (v2, `precond.cu`) separates Jacobian storage from inverse storage:

- `build_J_kernel`: called only on `jok=SUNFALSE`; reads y and its four neighbors, computes J_i, stores 9 doubles/cell. Cost comparable to one RHS evaluation.
- `build_Pinv_kernel`: called on *every* `psetup` regardless of `jok`; reads d_J and current γ, computes P_i^{-1} via Cramer's rule. Pure arithmetic, no stencil reads, negligible cost.

Results. The preconditioner was measured together with the fused NVector (both Tier 1 and Tier 3 active, Modified GS for the large grid). The initial implementation exposed a bug in the `jok` handling, which we corrected in v2. The final reported performance numbers use the corrected preconditioner together with the analytic JTV described next.

Configuration	Time (ms)	Speedup vs. baseline
Baseline	46,249	1.00×
Final optimized configuration (Fused + Precond v2 + JTV)	24,901	1.86×

Even before correcting the implementation, profiling showed that the block-diagonal preconditioner did not substantially reduce the GMRES iteration count per Newton step, which remained ≈5.5 and essentially unchanged from the unpreconditioned baseline. The 3×3 block Jacobi captures only the self-coupling terms. The dominant off-diagonal contribution is the exchange coupling $\partial H_i/\partial m_j = c_{\text{che}} = 4.0$ for each of the four neighbor cells, which is four times larger than the anisotropy self-term and is excluded from the block-diagonal approximation by construction. The speedup therefore does not appear to arise from substantially fewer GMRES iterations per Newton step. Based on the available statistics, the preconditioner changes the cost of the linear solve more than it changes the outer nonlinear/integration trajectory. In contrast, adding the analytic JTV leaves `nst`, `nfe`, `nni`, `ncfn`, and `netf` unchanged relative to the preconditioned run, while reducing `nsetups` from 523 to 155, indicating that its benefit comes primarily from eliminating finite-difference Jacobian–vector overhead rather than changing solver behavior.

To isolate what changed after correcting the implementation and adding JTV, we compare the outer-solver statistics below.

Configuration	nst	nfe	nsetups	nni	ncfn	netf
Fused + Precond v1 (jok bug; FD Jv)	3027	3756	523	3753	270	2
Fused + Precond v2 + JTV	3027	3756	155	3753	270	2

These nearly identical outer-solver statistics show that JTV does not change the integration trajectory; its benefit is to replace the finite-difference Jacobian–vector approximation with a direct kernel.

Optimization 3: Analytic Jacobian–Vector Product (JTV)

Motivation. With Fused NVector and Precond v1 active, Nsight shows `f_kernel_group_soa_periodic` called 20,489 times, while CVODE reports only `nfe` = 3,756 true RHS evaluations. The ≈16,700 excess calls arise from CVODE's default finite-difference Jacobian–vector approximation:

$$J(t, y)\, v \;\approx\; \frac{f(y + \varepsilon v) - f(y)}{\varepsilon},$$

which requires one additional $f()$ call per GMRES iteration. At $\approx 85\,\mu$s per RHS kernel, this contributes $16{,}700 \times 85\,\mu\text{s} \approx 1.42$ s of pure overhead with no accuracy benefit.

Design. We register an analytic Jacobian–vector callback via `CVodeSetJacTimes(cvode_mem, NULL, JtvProduct)` (`jtv.cu`). Fixing the current state y and differentiating with respect to a perturbation v, the variation of the effective field is:

$$\begin{aligned}
\delta h_1^{(i)} &= (c_{\text{che}} + c_{\text{chb}})(v_{1,L} + v_{1,R}) + c_{\text{che}}(v_{1,U} + v_{1,D}) \\
\delta h_2^{(i)} &= c_{\text{che}}(v_{2,L} + v_{2,R} + v_{2,U} + v_{2,D}) \\
\delta h_3^{(i)} &= c_{\text{che}}(v_{3,L} + v_{3,R} + v_{3,U} + v_{3,D}) + v_{3,i}
\end{aligned}$$

and the Jacobian–vector product at cell i is:

$$\begin{aligned}
(Jv)_{1,i} &= c_{\text{chg}}(v_{3,i} h_2 + m_3\, \delta h_2 - v_{2,i} h_3 - m_2\, \delta h_3) + c_\alpha(\delta h_1 - \delta\mu\, m_1 - \mu\, v_{1,i}) \\
(Jv)_{2,i} &= c_{\text{chg}}(v_{1,i} h_3 + m_1\, \delta h_3 - v_{3,i} h_1 - m_3\, \delta h_1) + c_\alpha(\delta h_2 - \delta\mu\, m_2 - \mu\, v_{2,i}) \\
(Jv)_{3,i} &= c_{\text{chg}}(v_{2,i} h_1 + m_2\, \delta h_1 - v_{1,i} h_2 - m_1\, \delta h_2) + c_\alpha(\delta h_3 - \delta\mu\, m_3 - \mu\, v_{3,i})
\end{aligned}$$

where $\mu = m \cdot H$ and $\delta\mu = v \cdot H + m \cdot \delta H$. This expression has exactly the same 4-neighbor stencil structure as the RHS kernel and is evaluated in a single 2D CUDA kernel launch with no $f()$ call, no finite-difference error, and no host transfer.

Results. JTV was deployed together with Precond v2 (corrected `jok`). The key effect of adding JTV is shown below:

Configuration	`f_kernel` calls	`jtv_kernel` calls	Time (ms)
Pre-JTV configuration	20,489	–	28,440
Final optimized configuration	3,655	16,957	24,901

The RHS kernel call count drops from 20,489 to 3,655, matching `nfe` and confirming complete elimination of finite-difference overhead. The 16,957 `jtv_kernel` calls replace those evaluations; each call costs $\approx 127\,\mu$s (vs. $\approx 85\,\mu$s for the RHS kernel), which is consistent with the additional reads required for the perturbation vector v.

This shows that the main value of JTV is not to change the outer integration trajectory, but to replace redundant finite-difference RHS evaluations with a direct analytic kernel.

Cumulative Effect and Remaining Bottleneck

Configuration (large grid, profiling clock)	Time (ms)	Speedup
Baseline	46,249	1.00×
Fused + CGS on large grid (*regression*)	103,321	0.45×
Fused (adaptive) + Precond v2 (jok fix) + JTV	**24,901**	**1.86**×
Small grid (600×128, neq = 76,800):		
Baseline	5,348	1.00×
Fused + Precond v2 + JTV (small grid)	847	**6.3**×

The CGS regression on the large grid is an important negative result: it demonstrates that a straightforward sync-reduction technique can *increase* runtime when the problem is bandwidth-limited and the optimization introduces cache thrashing. This motivated the adaptive GS selection and informed the broader conclusion that optimization strategies must be matched to the prevailing execution regime.

Nsight kernel breakdown (Fused + Precond v2 + JTV).

Kernel	GPU time share	Instances	Avg. (μs)
`psolve_kernel`	19.5%	20,693	226
`linearSumKernel` (SUNDIALS)	16.6%	101,032	39
`dotProdKernel` (SUNDIALS)	13.9%	86,294	39
`linearCombinationKernel` (SUNDIALS)	11.9%	13,719	208
`jtv_kernel`	9.0%	16,957	127
`prodKernel` (SUNDIALS)	7.1%	20,693	82
`f_kernel_group_soa_periodic`	1.3%	3,655	86

The four GMRES-related kernels (rows 1–4) account for ≈62% of GPU time. GMRES averages 20,693/3,753 ≈ 5.5 iterations per Newton step, identical to the un-preconditioned baseline. The 3×3 block preconditioner does not reduce the GMRES iteration count because the dominant off-diagonal Jacobian terms arise from exchange coupling to neighboring cells ($\partial H_i/\partial m_j = c_{\text{che}} = 4.0$, four times the anisotropy self-term), which are excluded from the block-diagonal approximation. Capturing exchange coupling would require an additive Schwarz preconditioner over spatial patches, but the memory cost of dense block storage at practical patch sizes ($\geq 4 \times 4$ cells) exceeds available GPU memory for the problem sizes studied.

Takeaway. Taken together, these optimizations achieve a 1.86× end-to-end speedup on the large grid and a 6.3× speedup on the small grid. More importantly, they confirm the main technical conclusion of this stage of the project: in a CVODE-driven GPU simulation, the dominant cost lies not in the user physics kernel alone, but in the repeated reductions, vector operations, and orchestration surrounding it.

The three optimizations attack different parts of this overhead: fewer GPU–CPU synchronization barriers (Fused NVector), lower linear-solver cost (preconditioning), and elimination of redundant RHS evaluations (analytic JTV). The CGS regression further shows that optimization strategy must match the prevailing execution regime; a technique that helps in an overhead-limited setting can become counterproductive in a bandwidth-limited one.

Progress Summary

So far, the project has produced a fully working GPU-resident 2D micromagnetic solver for the LLG equation using CUDA and SUNDIALS/CVODE. We first completed a regular-grid local-interaction solver and performed a systematic performance study covering data layout, block configuration, solver tolerance, problem-size scaling, and Nsight-based execution analysis. These experiments showed that solver-driven orchestration and repeated vector operations are major contributors to total runtime, often more important than the user RHS kernel itself.

We then extended the implementation to irregular masked geometries on the same structured grid. At this stage, we implemented and evaluated both a dense masked baseline and a compacted active-cell execution strategy, and showed that irregular geometry changes not only the GPU execution pattern but also CVODE's adaptive solver behavior. In parallel, we also explored solver-level optimization, including fused NVector operations, a problem-specific block preconditioner, and an analytic Jacobian–vector product. Together, these results give us a strong intermediate foundation for the remaining FFT and more irregular geometry work.

Progress vs. Proposal

Overall, the project remains aligned with the original proposal. The proposal stated that the project would combine CUDA, SUNDIALS/CVODE, and cuFFT, and would study both regular and irregular geometries while analyzing how data organization, execution strategy, and solver overhead affect end-to-end performance. That overall direction has not changed.

What has changed is the order of emphasis. Instead of integrating FFT immediately, we first pushed the local-only and irregular-geometry paths much further than originally planned, because profiling showed that solver overhead and synchronization were already dominant even before FFT was added. This was a useful pivot rather than a deviation: it allowed us to identify the real bottlenecks early, produce strong baseline and irregular results, and develop a clearer understanding

of what the FFT stage will need to overcome. The remaining major proposal-level deliverable is the FFT-based magnetostatic path, which we still plan to complete next, followed by experiments on more irregular geometries.

Revised Goals for the Poster Session

For the final stage of the project, our main goals are now:

- integrate the FFT-based magnetostatic computation path into the 2D solver,
- evaluate how adding FFT changes the runtime balance between local updates, global long-range computation, and solver overhead,
- extend the irregular geometry study beyond a single circular hole to at least one more irregular masked shape,
- compare how irregularity interacts with both dense masked and compacted execution strategies once FFT is enabled,
- prepare final plots and a clear end-to-end analysis for the poster and final report.

Our minimum success criterion remains the same as in the proposal: a correct GPU-resident 2D solver with one irregular geometry and one strong performance comparison. At this point, we have already exceeded that minimum on the local-only side, so the remaining work is focused on completing the FFT path and broadening the irregular-geometry study.

Plan for the Poster Session

At the poster session, we plan to show both performance graphs and qualitative simulation output. In particular, we expect to present:

- regular-grid performance plots for layout, block, tolerance, and problem-size scaling,
- irregular-geometry comparisons (dense masked vs. compacted active-cell execution),
- Nsight-based breakdowns showing how runtime shifts between the physics kernel, SUNDIALS vector operations, and solver overhead,
- visualizations of magnetization patterns for regular and irregular geometries,
- and, if integration is completed in time, a comparison between local-only and FFT-enabled runs.

The poster will therefore emphasize both systems analysis and physical simulation behavior rather than only one final speedup number.

Remaining Challenges and Concerns

The main remaining challenge is FFT integration. The original proposal identified FFT-based magnetostatics as a core part of the final system, but also noted that it adds packing/unpacking, temporary buffers, and extra synchronization across computational stages. We now know from the local-only study that solver overhead is already substantial, so the key question is whether the heavier FFT-based RHS will better amortize that overhead or instead introduce new bottlenecks.

A second concern is geometry complexity. So far we have shown one irregular masked geometry and one compacted execution strategy, which already produced useful results. However, more irregular masks may introduce stronger divergence, more inactive-cell imbalance, and more complex indexing behavior. This is not an unknown in the sense of "we do not know what to do," but it does mean that the final stage is still partly an experimental question: we need to see whether FFT and more irregular masks reinforce or change the bottlenecks identified so far.

Updated Project Schedule

- **Apr. 14–Apr. 17:** clean up the current local-only and irregular results, finalize milestone writeup, and stabilize the FFT code path interface with the existing 2D solver.
- **Apr. 17–Apr. 20:** complete integration of the FFT-based magnetostatic computation into the 2D solver; verify correctness on small regular-grid cases.
- **Apr. 20–Apr. 23:** run FFT-enabled experiments on regular grids; collect runtime breakdowns separating local work, FFT work, and solver overhead.
- **Apr. 23–Apr. 26:** extend the irregular-geometry study to at least one more irregular masked shape; compare dense and compacted execution with and without FFT where feasible.
- **Apr. 26–Apr. 28:** generate final plots, refine Nsight-based analysis, and decide which comparisons are strongest for the final report and poster.
- **Apr. 28–Apr. 30:** finish final report writing, prepare poster figures and visualizations, and polish the project web page.

References

1. Programming Massively Parallel Processors: A Hands-on Approach, 4th Edition, Morgan Kaufmann, 2022.
   `https://www.oreilly.com/library/view/programming-massively-parallel/9780323984638/?sso_link=yes&sso_link_from=cmu-edu`
2. LLNL, *SUNDIALS: Suite of Nonlinear and Differential/Algebraic Equation Solvers.*
   `https://github.com/LLNL/sundials`
3. NVIDIA Corporation, *CUDA Toolkit Documentation.*
   `https://docs.nvidia.com/cuda/`
4. NVIDIA Corporation, *cuFFT Library Documentation.*
   `https://docs.nvidia.com/cuda/cufft/`

5. NVIDIA Corporation, *Nsight Systems and Nsight Compute*.
`https://developer.nvidia.com/nsight-tools`

6. Y. Chen et al., "Incoherent Magnetic Switching of $L1_0$ FePt Grains,"
`https://ieeexplore.ieee.org/document/9269489`